LLMs & RAG
From Zero to Advanced

A practical, plain-English guide to how Large Language Models work, and how Retrieval-Augmented Generation makes them accurate.

10 CHAPTERS • LEVEL 0 → ADVANCED • BEGINNER FRIENDLY

Contents

01	What Is a Large Language Model? (Level 0)	L0
02	How LLMs Read Text: Tokens & Embeddings (Level 0)	L0
03	Inside the Transformer: Attention Explained (Level 1)	L1
04	How LLMs Are Trained (Level 1)	L1
05	Where LLMs Fall Short: Hallucination & Stale Knowledge (Level 1)	L1
06	What Is RAG? The Big Idea (Level 0)	L0
07	Anatomy of a RAG Pipeline (Level 2)	L2
08	Chunking, Embeddings & Vector Search in Practice (Level 2)	L2
09	Advanced RAG: Hybrid Search, Reranking & Agentic RAG (Level 3)	L3
10	Evaluating RAG Systems & a Glossary of Terms (Level 3)	L3

How to use this guide

Each chapter is tagged by difficulty: **L0** = no prior knowledge needed, **L1** = basic ML familiarity helpful, **L2** = hands-on / practitioner level, **L3** = advanced / production concerns. Read in order, or jump straight to what you need.

01 What Is a Large Language Model?

LEVEL 0

A **Large Language Model (LLM)** is a computer program trained on huge amounts of text (books, websites, code, conversations) to do one core task extremely well: **predict the next word** in a sentence, given everything written before it.

That sounds simple, but if a model gets very good at predicting the next word across billions of sentences, something surprising happens: it starts to look like it understands grammar, facts, reasoning, and even writing style — because predicting text well requires implicitly learning patterns about the world described in that text.

A simple mental model

- You type a **prompt** (a question or instruction).
- The model turns your words into numbers it can process.
- It predicts the most likely next word, adds it, then predicts the next one again — repeating this thousands of times to build a full response.
- This word-by-word generation is why responses sometimes appear to "stream" in on screen.

Everyday examples of LLMs

Product / Family	Made by	What it's commonly used for
Claude	Anthropic	Conversation, writing, coding, analysis
GPT (ChatGPT)	OpenAI	Conversation, writing, coding
Gemini	Google	Conversation, search integration
Llama	Meta	Open-weight model for research & apps

Key takeaway

An LLM doesn't "look things up" the way a search engine does. It generates answers from patterns it learned during training. That single fact explains almost every strength and weakness you'll read about in this guide.

02 How LLMs Read Text: Tokens & Embeddings

LEVEL 0

Computers don't understand words — they understand numbers. Before an LLM can process your prompt, it has to convert text into a numeric form in two steps: **tokenization**, then **embedding**.

Step 1: Tokenization

Text is broken into small chunks called **tokens**. A token is often a whole short word, part of a longer word, or a punctuation mark — roughly ¾ of a word on average in English.

```
Text: "Retrieval is powerful"
Tokens: ["Retrieval", " is", " power", "ful"]
IDs: [23991, 318, 1176, 913]
```

Step 2: Embeddings

Each token ID is converted into an **embedding** — a list of numbers (a vector), typically hundreds to thousands of values long. Similar meanings end up as similar vectors. This is the mathematical trick that lets a model treat "king" and "queen" as related, or "puppy" and "dog" as close in meaning, purely from numbers.

Why this matters for later chapters

Embeddings aren't just an internal detail of LLMs — they're also the core technology behind RAG's search step, which you'll meet in Chapter 6. The same idea (turning meaning into comparable numbers) powers both.

Beginner tip

Context window = the maximum number of tokens (input + output combined) a model can "see" at once. Longer conversations or documents eventually exceed this limit, which is one reason RAG exists — see Chapter 6.

03 Inside the Transformer: Attention Explained

LEVEL 1

Almost every modern LLM is built on an architecture called the **Transformer**, introduced in 2017. Its key innovation is a mechanism called **self-attention**.

What attention does

For every word, attention asks: "Which other words in this sentence should I pay the most attention to, in order to understand this word correctly?" It then blends information from those relevant words together.

Example: in "The trophy didn't fit in the suitcase because **it** was too big," attention lets the model figure out that "it" most likely refers to "the trophy," not "the suitcase" — by weighing the relationships between all the words at once, not just reading left to right.

Why this replaced older approaches

Approach	How it processes text	Main limitation
RNN / LSTM (older)	Reads one word at a time, in order	Struggles with long-range connections; slow to train
Transformer (current)	Looks at all words simultaneously via attention	Needs more compute, but scales extremely well

Stacking it all together

- A Transformer stacks dozens of attention + processing layers on top of each other.
- Each layer refines the model's understanding of the text a little more.
- "Parameters" (the billions in a model's name, e.g. an 8B or 70B model) are the learned numerical weights inside these layers.
- More parameters generally means more capacity to capture complex patterns — but also more compute cost to run.

04 How LLMs Are Trained

LEVEL 1

Training a modern LLM happens in stages. Each stage shapes the model's behavior in a different way.

1. Pretraining

The model reads enormous amounts of text (a large slice of the internet, books, code) and learns to predict the next token. This is where it absorbs grammar, facts, and reasoning patterns. Extremely expensive: often months of computation on thousands of specialized chips.

2. Supervised Fine-Tuning (SFT)

The base model is shown examples of good instruction-following — question paired with an ideal answer — to teach it to behave like a helpful assistant rather than just an autocomplete engine.

3. Reinforcement Learning from Human Feedback (RLHF)

Humans rank multiple model responses from best to worst. The model is further adjusted to produce responses people prefer — more helpful, honest, and safe.

4. (Optional) Task-specific fine-tuning

Organizations sometimes further adjust a model on their own narrow data, e.g. legal documents or customer support transcripts, to specialize its behavior.

Fine-tuning vs. RAG — a preview

Fine-tuning changes the model's internal weights and is good for teaching *style or skill*. RAG (starting Chapter 6) leaves the model unchanged and instead feeds it fresh information at question time — better for teaching *facts and current knowledge*. Chapter 9 compares them directly.

05 Where LLMs Fall Short

LEVEL 1

Understanding LLM limitations is essential — it's the entire motivation for RAG, which starts in the next chapter.

1. Knowledge cutoff

A model only knows what existed in its training data up to a certain date. It has no built-in awareness of anything that happened afterward, unless it's given that information directly in the conversation.

2. Hallucination

Because an LLM generates the statistically likely next word rather than retrieving verified facts, it can produce plausible-sounding but incorrect statements — fabricated citations, wrong dates, invented function names — stated with total confidence.

3. No access to private or proprietary data

A general-purpose LLM was never trained on your company's internal wiki, your personal files, or yesterday's news. It simply has no way to know that content exists.

4. Limited context window

Even very long context windows have a ceiling. You can't just paste an entire company knowledge base into every prompt.

Limitation	Root cause	Common fix
Knowledge cutoff	Fixed training data snapshot	RAG / live tool use
Hallucination	Generates plausible text, not verified facts	RAG + grounding, citations
No private data	Never trained on it	RAG over your own documents
Context limits	Finite input size	Retrieve only relevant chunks (RAG)

06 What Is RAG? The Big Idea

LEVEL 0

Retrieval-Augmented Generation (RAG) is a technique that gives an LLM access to external, up-to-date, or private information at the moment it answers a question — instead of relying only on what it memorized during training.

The two halves of the name

- **Retrieval**: before answering, the system searches a knowledge base (documents, a database, a website) for the pieces of text most relevant to the question.
- **Augmented Generation**: those retrieved pieces of text are inserted into the prompt alongside the question, so the LLM generates its answer *using* that fresh material.

A simple analogy

Open-book exam vs. closed-book exam

A plain LLM answers like a student taking a **closed-book exam** — relying purely on memory. A RAG system lets the same student take an **open-book exam** — first flipping to the relevant page, then writing an answer based on what's actually written there. The student (the LLM) still does the writing, but the source material keeps them accurate.

Why RAG matters

- Keeps answers current without retraining the whole model.
- Lets an LLM answer questions about private or internal documents it never saw during training.
- Reduces hallucination, because the model is grounded in retrieved text.
- Can cite sources, since you know exactly which document chunks were used.
- Is far cheaper and faster to update than fine-tuning — add a new document instead of retraining.

07 Anatomy of a RAG Pipeline

LEVEL 2

A production RAG system has two distinct phases: an offline **indexing** phase (done once, or whenever documents change) and an online **query** phase (done every time a user asks a question).

Phase A — Indexing (offline, done in advance)

- **1. Collect** source documents (PDFs, web pages, tickets, wikis).
- **2. Chunk** each document into smaller passages (see Chapter 8).
- **3. Embed** each chunk into a vector using an embedding model.
- **4. Store** the vectors, plus the original text and metadata, in a **vector database** (e.g. Pinecone, Weaviate, Chroma, pgvector).

Phase B — Query time (online, per user question)

- **1. Embed the question** using the same embedding model used during indexing.
- **2. Search** the vector database for the chunks whose vectors are most similar to the question's vector ("nearest neighbor" search).
- **3. Assemble a prompt** that combines the user's question with the retrieved chunks, usually with instructions like "Answer using only the context below."
- **4. Generate** the final answer with the LLM, ideally citing which chunk(s) supported each claim.

End-to-end flow at a glance

```
User question
-> Embed question
-> Search vector DB for top-k similar chunks
-> Build prompt: [instructions] + [retrieved chunks] + [question]
-> LLM generates grounded answer
-> Answer (+ citations) returned to user
```

Common vector databases

Pinecone, Weaviate, Qdrant, Milvus, Chroma, and Postgres with the pgvector extension are widely used choices, each trading off scale, hosting model, and ease of setup differently.

08 Chunking, Embeddings & Vector Search

LEVEL 2

The quality of a RAG system depends heavily on decisions made long before the LLM is ever called. This chapter covers the three most consequential ones.

1. Chunking strategy

Documents must be split into passages small enough to embed meaningfully and fit in a prompt, but large enough to retain context.

Strategy	How it works	Best for
Fixed-size chunks	Split every N tokens, often with overlap	Simple, fast baseline
Sentence / paragraph based	Split on natural language boundaries	Preserves readability
Semantic chunking	Split where topic/meaning shifts	Higher quality, more compute
Document-structure aware	Split by headings, sections, tables	Structured docs (manuals, contracts)

A common default: 200–500 token chunks with roughly 10–20% overlap between neighboring chunks, so that ideas split across a boundary aren't lost.

2. Choosing an embedding model

Embedding models vary in vector size, language coverage, and whether they're general-purpose or domain-tuned (e.g. legal or biomedical). The embedding model used for indexing and the one used for queries must match, or similarity scores become meaningless.

3. Similarity search

Vector databases typically compare vectors using **cosine similarity** or **dot product**, and use approximate nearest-neighbor algorithms (like HNSW) to search millions of vectors in milliseconds instead of comparing against every single one.

> **Practitioner tip**
>
> Retrieval quality problems are the #1 cause of bad RAG answers — more often than the LLM itself. If answers seem wrong, inspect what was actually retrieved before assuming the model is at fault.

09 Advanced RAG Techniques

LEVEL 3

Basic "embed, search, stuff into prompt" RAG works, but production systems layer on additional techniques to improve accuracy and handle harder questions.

Hybrid search

Combines vector (semantic) search with traditional keyword search (BM25). Catches cases where exact terms — product codes, names, acronyms — matter more than semantic similarity.

Reranking

After an initial retrieval pulls, say, 50 candidate chunks, a separate, more expensive "reranker" model re-scores and reorders them for relevance, and only the top few are kept for the prompt. Improves precision at modest extra cost.

Query rewriting / expansion

The user's raw question is rewritten by an LLM into one or more better search queries before retrieval — useful when the original phrasing is vague, conversational, or depends on earlier chat context.

Multi-hop / multi-step retrieval

Complex questions requiring multiple facts trigger several rounds of retrieval, where each round's results inform the next search, rather than one single retrieval pass.

Agentic RAG

An LLM acts as an agent that decides *whether* to retrieve, *what* to search for, and whether the retrieved evidence is sufficient — looping and re-searching if not, rather than following one fixed pipeline.

Contextual / metadata filtering

Retrieval is narrowed using metadata (date, department, document type, access permissions) before or alongside the vector search.

RAG vs. fine-tuning vs. long context — the honest comparison

RAG is best for facts that change often or are private. **Fine-tuning** is best for teaching tone, format, or a specialized skill. **Long-context prompting** (pasting documents directly) is best for a handful of documents in a single session. Many production systems combine all three.

10 Evaluating RAG Systems & Glossary

LEVEL 3

You can't improve what you don't measure. RAG evaluation typically looks at retrieval quality and generation quality separately.

Retrieval metrics

Metric	What it measures
Precision@k	Of the top-k retrieved chunks, what fraction are actually relevant?
Recall@k	Of all relevant chunks that exist, what fraction were retrieved in the top-k?
MRR (Mean Reciprocal Rank)	How high up the first relevant result ranks, on average

Generation metrics

- **Faithfulness / groundedness** — does the answer only state things supported by the retrieved context, with no fabrication?
- **Answer relevance** — does the answer actually address the user's question?
- **Context precision/recall** — frameworks like RAGAS combine retrieval and generation scoring into one pipeline, often using an LLM itself as the judge.

Common pitfalls checklist

- Chunks too large → irrelevant text dilutes the prompt.
- Chunks too small → lost context, fragmented meaning.
- Mismatched embedding models between indexing and querying.
- No reranking → noisy top-k results reach the LLM.
- No citation / source tracking → impossible to verify answers.
- Stale index → documents updated but never re-embedded.

Quick glossary

Term	Meaning
Token	A small unit of text (word/sub-word) the model processes
Embedding	A numeric vector representing the meaning of text
Vector database	A database optimized for storing and searching embeddings
Context window	Max tokens a model can process in one request
Hallucination	A confident but factually incorrect model output

Term	Meaning
Chunking	Splitting documents into smaller retrievable passages
Reranking	Re-scoring retrieved results for relevance before use
Grounding	Basing an answer on retrieved evidence, not just memory

You've reached advanced level

You now understand how LLMs turn text into predictions, why they hallucinate and go stale, and how RAG — from basic pipelines to hybrid search, reranking, and agentic retrieval — closes that gap. The natural next step is building a small RAG prototype over your own documents.